

Day 05: ログイン画面のUIを作ろう

今日のゴール

Material-UI (MUI) を使って、美しいログイン画面を作ります。メールアドレスとパスワードの入力欄、ログインボタンを配置して、プロフェッショナルなUIを実装します。

【スクリーンショット: 完成したログイン画面】

なぜこれを作るのか？

ログイン機能は、ほとんどのWebアプリに必要な基本機能です。まずはUIを作ることで、Reactのコンポーネントやイベントハンドリングを実践的に学べます。また、MUIを使うことで、デザインの知識がなくても美しいUIを作れます。

例え話: 店の入り口が汚かったら、中に入りたくなくなりますよね。ログイン画面は、アプリの「玄関」です。第一印象が大事なので、見た目にもこだわります。

実装ステップ一覧

ステップ	作業内容	所要時間
Step 1	ログインページを作成	10分
Step 2	フォームコンポーネントを作成	15分
Step 3	バリデーションを追加	15分
Step 4	デザインを調整	10分

合計時間: 約50分

Step 1: ログインページを作成（10分）

ゴール: `src/app/login/page.tsx`を作成して、ログインページのベースを作ります。

初心者向け解説: Next.jsでは、`src/app`フォルダ内にフォルダを作成すると、自動的にルートが作成されます。`login`フォルダを作って、その中に`page.tsx`を作成すると、`/login`でアクセスできるようになります。

実装:

```
// filepath: src/app/login/page.tsx
export default function LoginPage() {
  return (
    <div>
      <h1>ログイン</h1>
    </div>
  );
}
```

コード解説:

コード	意味	例え
<code>export default</code>	このコンポーネントをエクスポート	商品を店頭に並べる
<code>LoginPage</code>	コンポーネント名	商品の名前
<code><h1></code>	見出しタグ	看板

確認ポイント:

- ブラウザで `http://localhost:3000/login` にアクセス
- 「ログイン」という見出しが表示される
- これでログインページの作成が完了です

【スクリーンショット: /loginにアクセスした画面】

学んだこと: Next.jsのApp Routerで、新しいページを作成できるようになりました。

Step 2: フォームコンポーネントを作成（15分）

ゴール: MUIのTextField、Buttonを使って、ログインフォームを作成します。

初心者向け解説: `TextField` は入力欄、`Button` はボタンを表すMUIのコンポーネントです。これらを組み合わせることで、美しいフォームを簡単に作れます。`useState` を使って、入力内容を管理します。

実装:

```
// filepath: src/app/login/page.tsx
'use client';

import { useState } from 'react';
import { Box, TextField, Button, Typography } from '@mui/

export default function LoginPage() {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');

  const handleSubmit = (e: React.FormEvent) => {
    e.preventDefault();
    console.log('Email:', email);
    console.log('Password:', password);
  };

  return (
    <Box>
      <Typography variant="h4">ログイン</Typography>
      <form onSubmit={handleSubmit}>
        <TextField
          label="メールアドレス"
          type="email"
          value={email}
          onChange={(e) => setEmail(e.target.value)}
        />
        <TextField
          label="パスワード"
          type="password"
          value={password}
          onChange={(e) => setPassword(e.target.value)}
        />
        <Button type="submit" variant="contained">
          ログイン
        </Button>
      </form>
    </Box>
  );
}
```

```
    </Box>
  );
}
```

コード解説:

コード	意味	例え
<code>'use client'</code>	クライアントコンポーネント宣言	「このページは動的です」と宣言
<code>useState</code>	状態管理フック	ゲームのセーブデータ
<code>TextField</code>	入力欄	お店の注文用紙
<code>Button</code>	ボタン	お店のレジのボタン

確認ポイント:

1. メールアドレスとパスワードの入力欄が表示される
2. 入力すると、コンソールにログが表示される
3. これでフォームの作成が完了です

【スクリーンショット: ログインフォームの画面】

学んだこと: MUIのコンポーネントを使って、入力フォームを作成できるようになりました。

Step 3: バリデーションを追加（15分）

ゴール: 入力内容のバリデーション（検証）を追加します。

初心者向け解説: バリデーションは、ユーザーが正しい形式で入力しているかをチェックする機能です。メールアドレスが正しい形式か、パスワードが入力されているかなどをチェックします。エラーメッセージを表示することで、ユーザーに修正を促します。

実装:

```
// filepath: src/app/login/page.tsx
'use client';

import { useState } from 'react';
import { Box, TextField, Button, Typography } from '@mui/

export default function LoginPage() {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [errors, setErrors] = useState({ email: '', passw

  const validate = () => {
    const newErrors = { email: '', password: '' };

    if (!email) {
      newErrors.email = 'メールアドレスを入力してください';
    } else if (!/\S+@\S+\.\S+/.test(email)) {
      newErrors.email = 'メールアドレスの形式が正しくありません
    }

    if (!password) {
      newErrors.password = 'パスワードを入力してください';
    }

    setErrors(newErrors);
    return !newErrors.email && !newErrors.password;
  };

  const handleSubmit = (e: React.FormEvent) => {
    e.preventDefault();
    if (validate()) {
      console.log('ログイン成功:', { email, password });
    }
  };

  return (
```

```

<Box>
  <Typography variant="h4">ログイン</Typography>
  <form onSubmit={handleSubmit}>
    <TextField
      label="メールアドレス"
      type="email"
      value={email}
      onChange={(e) => setEmail(e.target.value)}
      error={Boolean(errors.email)}
      helperText={errors.email}
    />
    <TextField
      label="パスワード"
      type="password"
      value={password}
      onChange={(e) => setPassword(e.target.value)}
      error={Boolean(errors.password)}
      helperText={errors.password}
    />
    <Button type="submit" variant="contained">
      ログイン
    </Button>
  </form>
</Box>
);
}

```

コード解説:

コード	意味	例え
<code>validate()</code>	バリデーション関数	注文内容をチェック
<code>error={Boolean(errors.email)}</code>	エラー状態を設定	赤いランプをつける
<code>helperText={errors.email}</code>	エラーメッセージ表示	「ここが間違ってます」と教える

確認ポイント:

- 1. 空欄でログインボタンを押すと、エラーメッセージが表示される
- 2. 正しい形式で入力すると、エラーが消える
- 3. これでバリデーションの追加が完了です

【スクリーンショット: バリデーションエラーが表示された画面】

学んだこと: 入力内容をバリデーションして、ユーザーに適切なフィードバックを返せるようになりました。

Step 4: デザインを調整 (10分)

ゴール: レイアウトを調整して、見栄えを良くします。

初心者向け解説: MUIの `Box` コンポーネントを使って、余白や中央揃えを設定します。 `sx` プロパティを使うことで、インラインでスタイルを指定できます。

実装:

```
// filepath: src/app/login/page.tsx (一部抜粋)
<Box
  sx={{
    maxWidth: 400,
    mx: 'auto',
    mt: 8,
    p: 3,
  }}
>
  <Typography variant="h4" sx={{ mb: 3 }}>
    ログイン
  </Typography>
  <form onSubmit={handleSubmit}>
    <TextField
      fullWidth
      sx={{ mb: 2 }}
      label="メールアドレス"
      type="email"
      value={email}
      onChange={(e) => setEmail(e.target.value)}
      error={Boolean(errors.email)}
      helperText={errors.email}
    />
    <TextField
      fullWidth
      sx={{ mb: 2 }}
      label="パスワード"
      type="password"
      value={password}
      onChange={(e) => setPassword(e.target.value)}
      error={Boolean(errors.password)}
      helperText={errors.password}
    />
    <Button fullWidth type="submit" variant="contained">
      ログイン
    </Button>
```

```
</form>
</Box>
```

スタイル解説:

プロパティ	意味	値の例
<code>maxWidth</code>	最大幅	400px
<code>mx: 'auto'</code>	左右の余白を自動（中央揃え）	-
<code>mt: 8</code>	上の余白	$8 \times 8\text{px} = 64\text{px}$
<code>p: 3</code>	内側の余白	$3 \times 8\text{px} = 24\text{px}$

確認ポイント:

- ログインフォームが画面中央に表示される
- 余白が適切に設定されている
- これでデザインの調整が完了です

【スクリーンショット: デザイン調整後のログイン画面】

学んだこと: MUIの `SX` プロパティを使って、レイアウトを調整できるようになりました。

今日のまとめ

- ☐ ログインページを作成できた
- ☐ MUIのTextField、Buttonを使えた
- ☐ useState で入力内容を管理できた
- ☐ バリデーションを実装できた
- ☐ デザインを調整できた

⚠ つまづきポイント

エラー/問題	原因	解決方法
<code>'use client'</code> エラー	クライアントコンポーネント宣言が必要	ファイル先頭に <code>'use client'</code> を追加
MUIコンポーネントが表示されない	インポートが間違っている	<code>@mui/material</code> からインポート
バリデーションが動かない	<code>validate()</code> が呼ばれていない	<code>handleSubmit</code> 内で <code>validate()</code> を呼ぶ

次回予告

Day 6では、ユーザー登録画面を作ります。ログイン画面と同じように、MUIを使って美しいUIを実装します。